If you were performing a chemistry experiment, you'd probably care about the temperature and composition of your sample, but not the arrangement of the chairs in the hall outside. The same principle applies to your software experiments. When you set up the test environment for a trip planner, you may need to control the time zone but not the low-level database details. Test doubles help you control just the factors you care about.

Let's make these ideas more concrete with an example: testing the signup process for a web app. Like many such apps, this one requires new passwords to meet a minimum strength standard. Here's what our strength validator looks like:

15-using-test-doubles-effectively/01/password_strength_validator/lib/password_strength_validator.rb

```
class PasswordStrengthValidator
  def strong_enough?
    return false unless password.length >= Acme::Config.min_password_length

    # ... more validations ...
  end
end
```

To check this code, we might write a couple of specs like the following:

15-using-test-doubles-effectively/01/password_s ... lidator/spec/password_strength_validator_spec.rb

```
RSpec.describe PasswordStrengthValidator do
  it 'rejects passwords shorter than 8 characters' do
    validator = PasswordStrengthValidator.new('a8E^rd2')
    expect(validator.strong_enough?).to eq false
  end

  it 'accepts passwords 8 characters or longer' do
    validator = PasswordStrengthValidator.new('a8E^rd2i')
    expect(validator.strong_enough?).to eq true
  end
end
```

Assuming the web app is configured to require eight-character passwords, these two test cases will exercise both sides of the conditional. But if we later reconfigure the app to require twelve characters (perhaps to meet a new company guideline), one of the specs will start to fail:

```
$ rspec
.F

Failures:

  1) PasswordStrengthValidator accepts passwords 8 characters or longer
     Failure/Error: expect(validator.strong_enough?).to eq true

       expected: true
            got: false
```